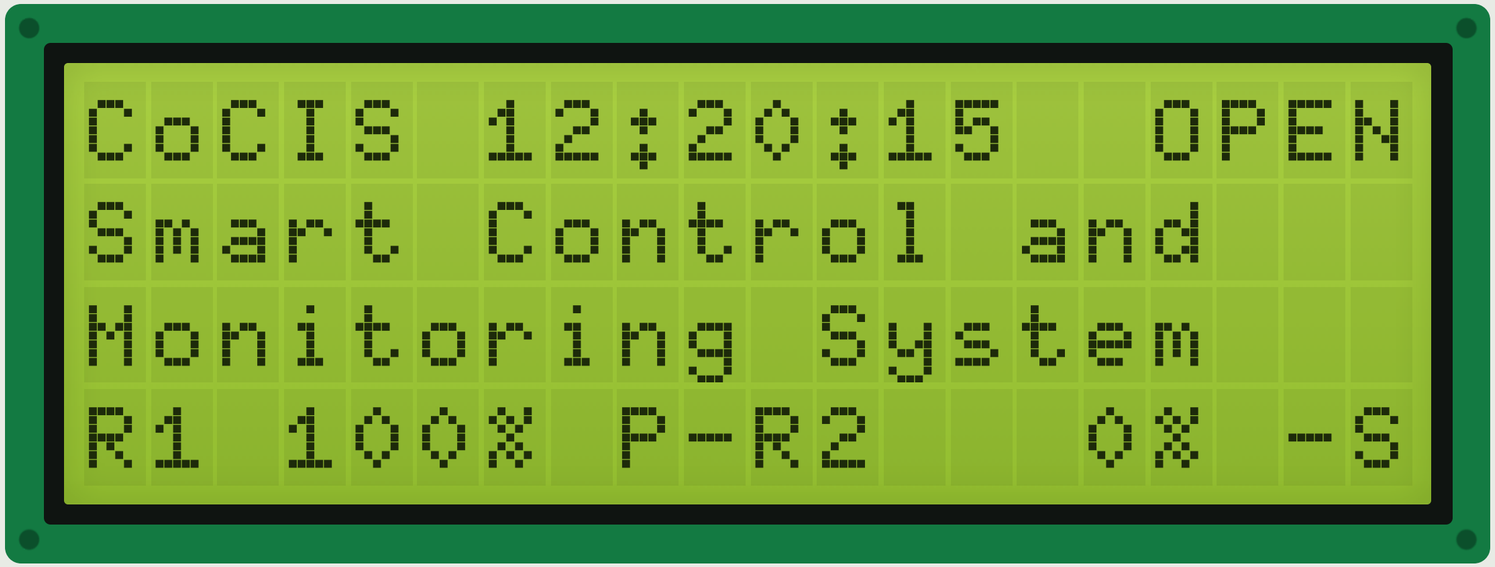


Group 15

Embedded Systems, 2026/2027.
College of Computing and Information Sciences, Makerere University.

Access control, occupancy records and automatic lighting for the CoCIS main entrance and two lecture rooms.

1
Mujuni Innocent
Overview, integration, storage and synchronisation

2
Martha Namuyimbwa
Entrance access control

3
Ochen Felix
Lecture-room lighting

4
Nakimuli Jollyne Flavia
Management console and admin login

5
Tamara Alice Nazziwa
Testing and cloud dashboard

Two problems, one pilot

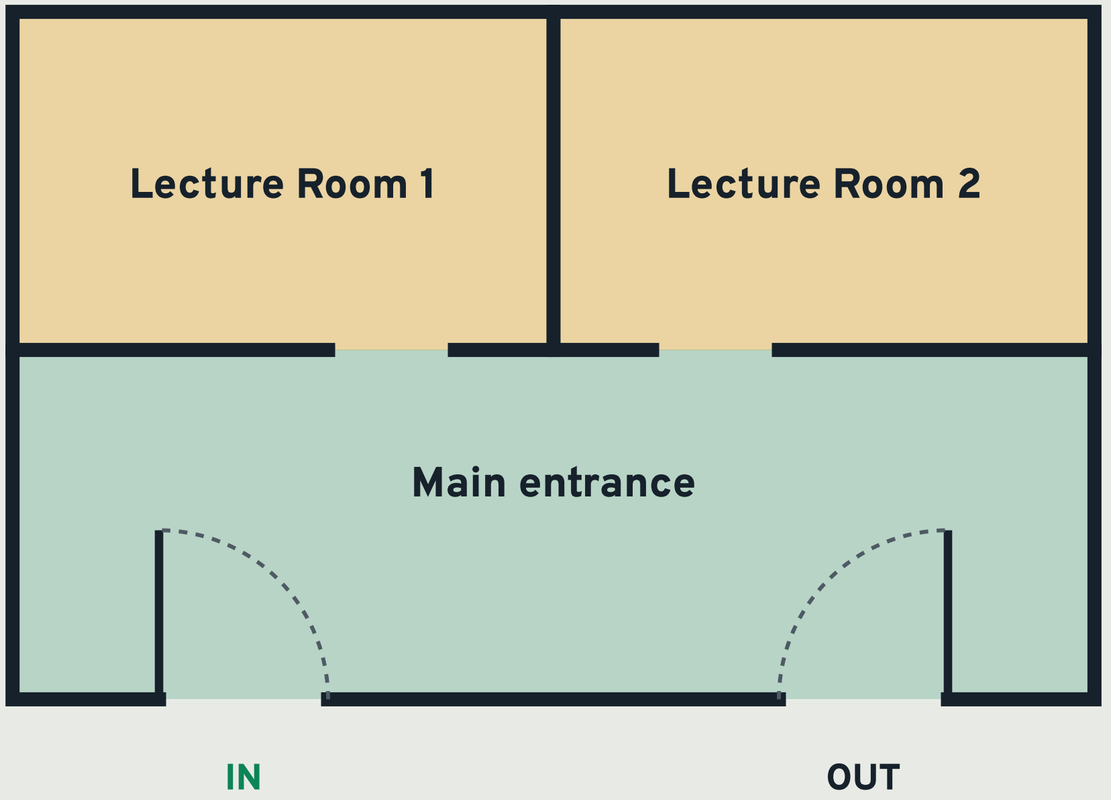
Valuable equipment

The college needs to control who comes in, who leaves, and who is still inside.

High electricity bills

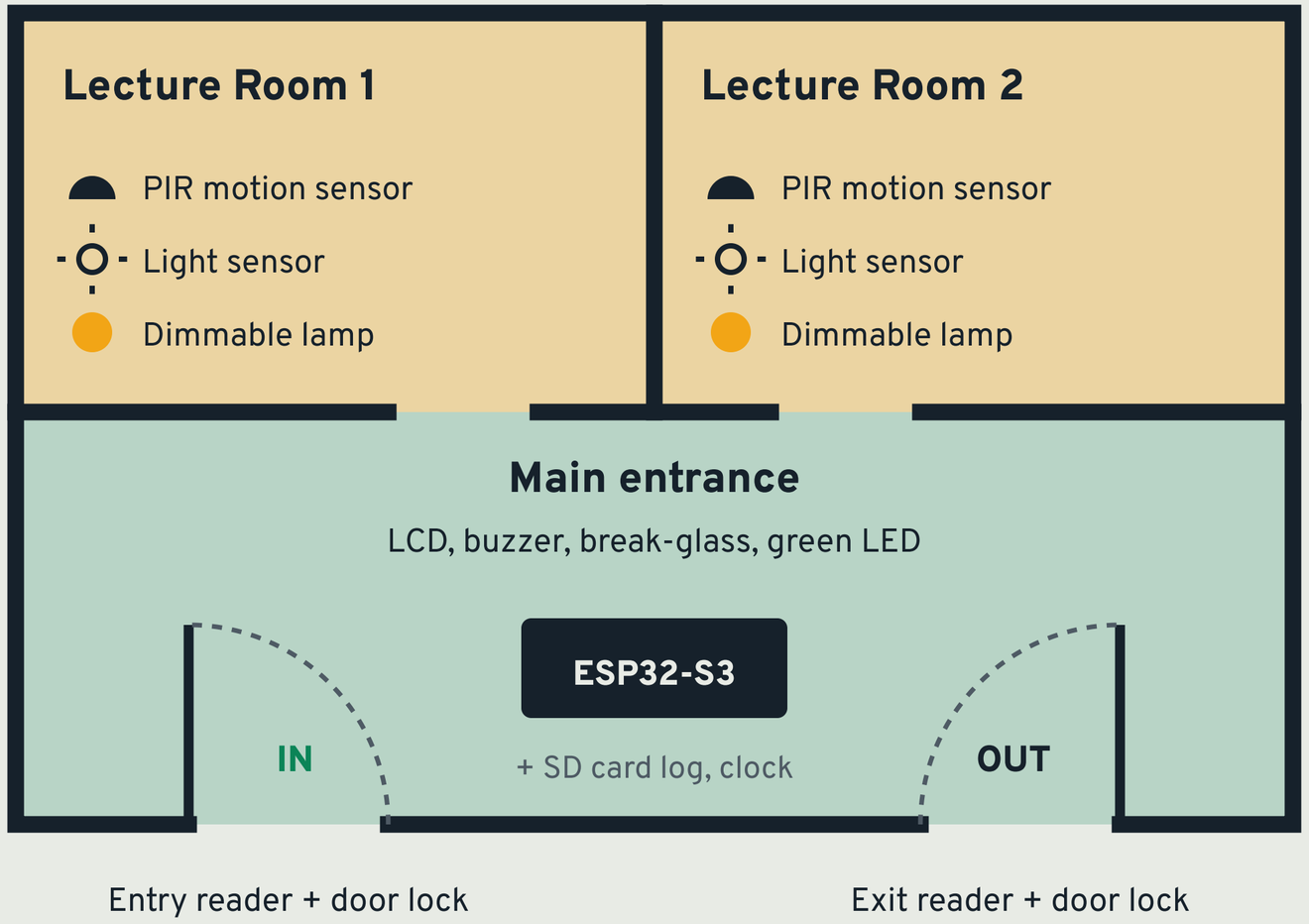
Lights stay on in lecture rooms that nobody is using.

Management asked for a pilot on the main entrance and two lecture rooms, run by ESP32 terminals.



Mujuni Innocent 24/U/07155/PS
Overview, integration, storage and synchronisation

Three terminals on one ESP32-S3



Main entrance hub

Two RFID readers and two servo door locks check people at entry and exit, keep the list of who is inside, and enforce opening hours.

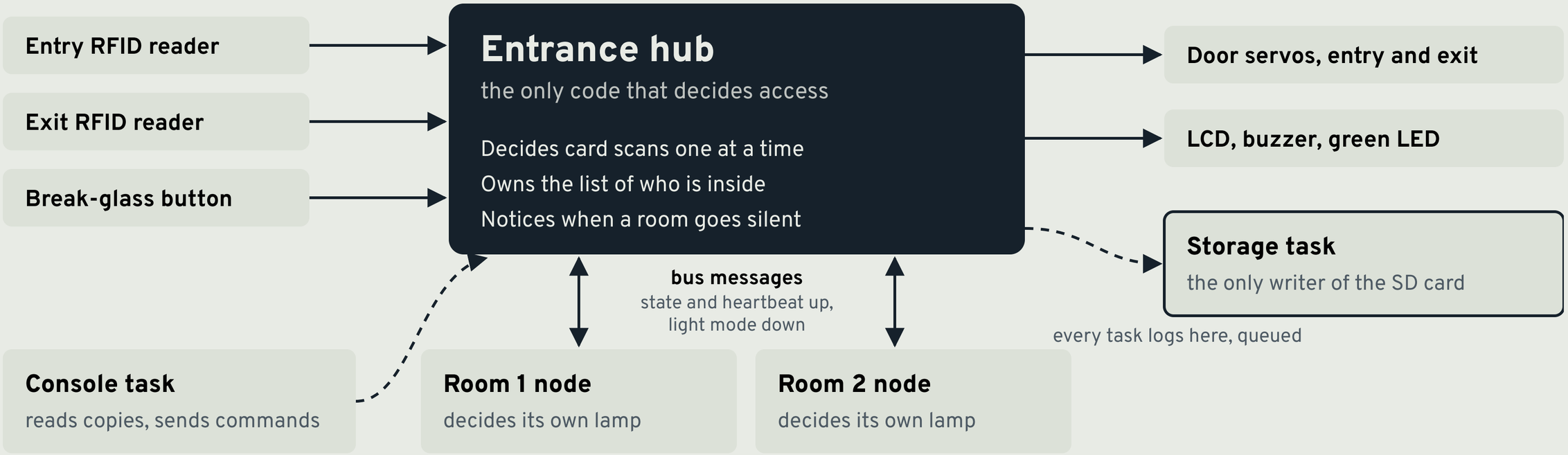
Lecture Rooms 1 and 2

Motion and light sensors switch and dim each lamp, use daylight, and report the energy saved.

Runs offline

No network needed. Built with ESP-IDF and FreeRTOS, simulated in Wokwi.

One authority, and everything starts in order



Start-up order
`main.c`

1	2	3	4	5	6	7
Flash storage (NVS)	Clock from the RTC	Event log, SHA-256 self-test	Users, schedule, who is inside	Bus inboxes, light sensors	Hub and two room tasks	Serial console

Every shared thing has one owner

Who is inside, and every access decision	Only the hub task writes it. Scans wait in its inbox and are decided one at a time.
The SD card	Only the storage task touches it. Every other task queues its log lines.
Users, schedule, login state	One mutex each, held for a quick copy. Readers get a copy, never a pointer.
Settings and the ledger in flash	Saved to NVS with an atomic commit, so the list of people inside survives a power cut.
Messages on the bus	A sequence number on every message. The receiver drops repeats and replays.

```
bool bus_send(src, dst, msg, wait)
{
    xSemaphoreTake(s_send_lock[src], ...);
    msg→src = src;
    msg→seq = s_next_seq[src] + 1;
    bool sent = xQueueSend(s_inbox[dst],
                           msg, wait);
    if (sent)
        s_next_seq[src] = msg→seq;
    xSemaphoreGive(s_send_lock[src]);
    return sent;
}
```

bus.c, simplified. The per-sender lock makes "take the next number, queue the message" one step. We found that race in code review and fixed it.

Mujuni Innocent 24/U/07155/PS
Overview, integration, storage and synchronisation



The log notices if anyone edits it

```
000008|2026-10-28T11:35:49|ACCESS|uid=01020304 name=Alice side=IN ok=1 reason=OK |e9761b4b…
000009|2026-10-28T11:35:51|ACCESS|uid=01020304 name=Alice side=IN ok=0 reason=ANTI_PASSBACK ok=1 reason=OK |e9037d46…
000010|2026-10-28T11:35:53|ACCESS|uid=01020304 name=Alice side=OUT ok=1 reason=OK |18953a92…
000011|2026-10-28T11:35:54|ACCESS|uid=C0FFEE99 name=unknown side=IN ok=0 reason=UNKNOWN_CARD |3e562ebe…
```

```
hash = SHA-256( previous hash | line )
```

One file per day on the SD card. Every line is saved with `fsync`. With no card, the log is kept in RAM.

```
cocis> log verify
TAMPERED: hash chain broken at line 9 (of 9 checked).
```




One tap at the entry reader



Alice

Student
`01020304`

The Blue card in the Wokwi reader's panel. Tap it and the LCD shows this.

1. The reader task reads the card's UID with our MFRC522 driver and sends `CARD_SCAN` to the hub.
2. The hub looks the card up: Alice, student, enabled.
3. It gathers the facts: is she inside, is the building open, is there a lockdown?
4. `access_decide()` returns the verdict: `OK`
5. She is marked inside, and only the entry door opens, for 3 seconds.
6. An `ACCESS` line goes to the log; the LCD and buzzer tell her.



The rules are one function: the first match wins

```
if (!req→known)     return ACCESS_UNKNOWN_CARD;

if (!req→enabled)   return ACCESS_DISABLED;

if (entering while inside || leaving while outside)

                     return ACCESS_ANTI_PASSBACK;

if (req→lockdown)   return privileged ? OK_PRIVILEGED : LOCKDOWN;

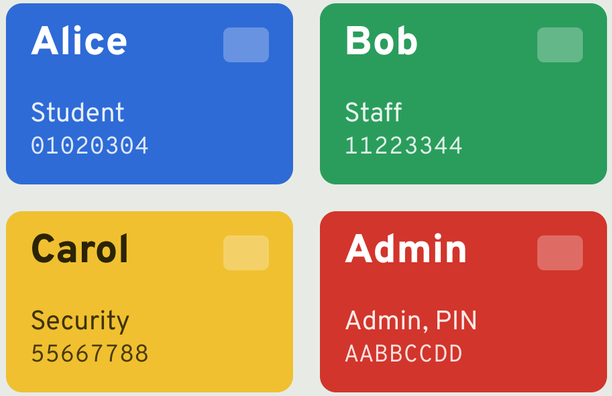
if (req→schedule_open)   return ACCESS_OK;

if (privileged)     return ACCESS_OK_PRIVILEGED;

if (leaving && req→escort_active)   return ACCESS_OK_ESCORT;

return ACCESS_OUTSIDE_HOURS;
```

Anti-passback means one card can't let two people in: no entering twice without leaving.



Security and admin may pass at any hour. No hardware calls, so it is unit tested on a PC.

Martha Namuyimbwa 24/U/09436/PS
Entrance access control



After hours: held inside, never trapped

- **Bob tries to leave after hours: refused.**
  `ACCESS name=Bob role=STAFF side=OUT ok=0 reason=OUTSIDE_HOURS`
- **Security is told someone is waiting.**
  `ALERT code=AFTER_HOURS_EXIT detail=Bob`
- **Carol from security leaves: a 60 s escort window opens.**
  `ACCESS name=Carol role=SECURITY side=OUT ok=1 reason=OK_PRIVILEGED`
- **Bob leaves with her, and the exit is recorded.**
  `ACCESS name=Bob role=STAFF side=OUT ok=1 reason=OK_ESCORT`

Martha's code: `drivers/mfrc522.c` `nodes/rfid.c` `core_logic/access_policy.c` `nodes/hub_access.c` `nodes/door.c`



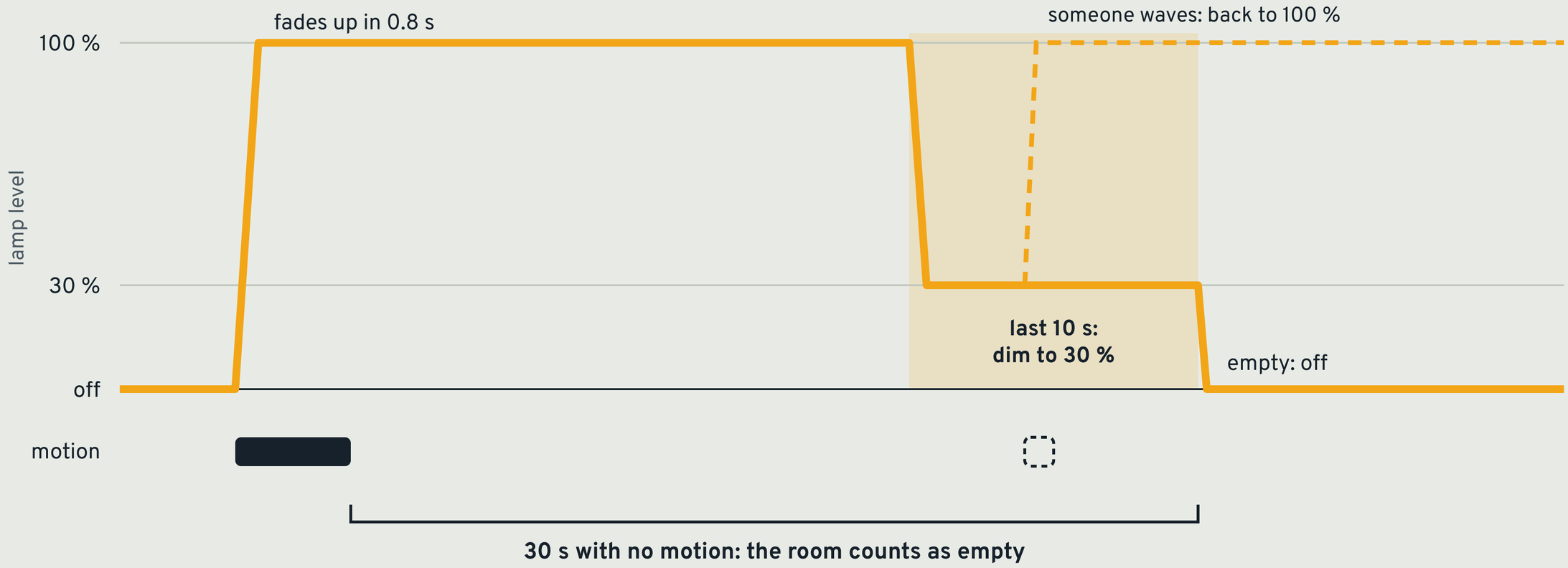
Break-glass overrides every rule

Both doors are held open and an alert is logged. Only an admin can clear it.

Martha Namuyimbwa 24/U/09436/PS
Entrance access control

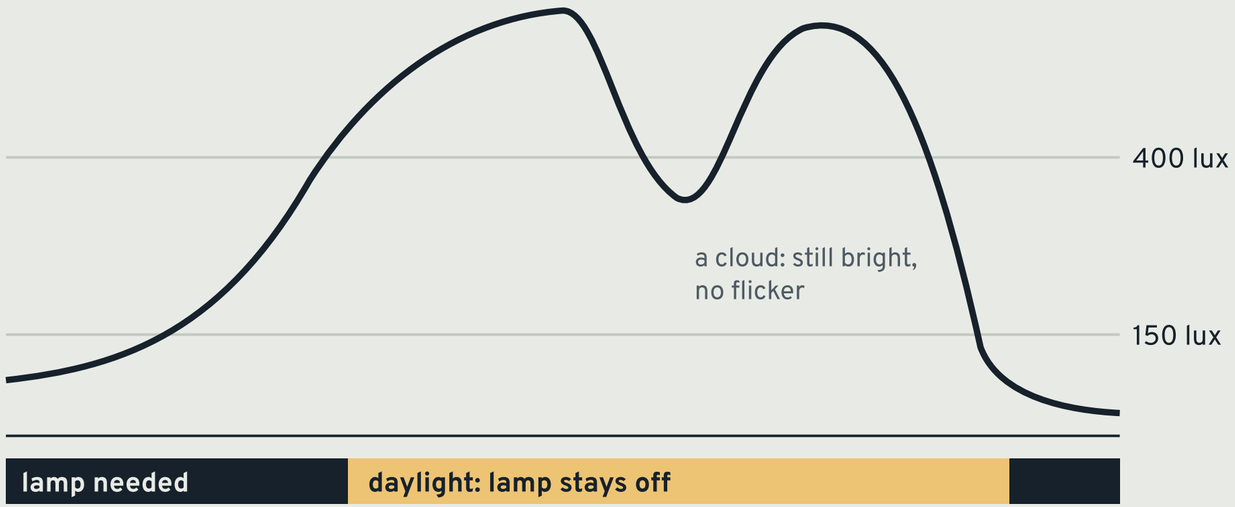


The lights follow the people in the room





Daylight harvesting, and the energy it saves




```
if (lux >= cfg→bright_lux)      r→daylight = true;
else if (lux <= cfg→dark_lux)   r→daylight = false;
/* in between: keep the last decision */
```

saved = baseline − used

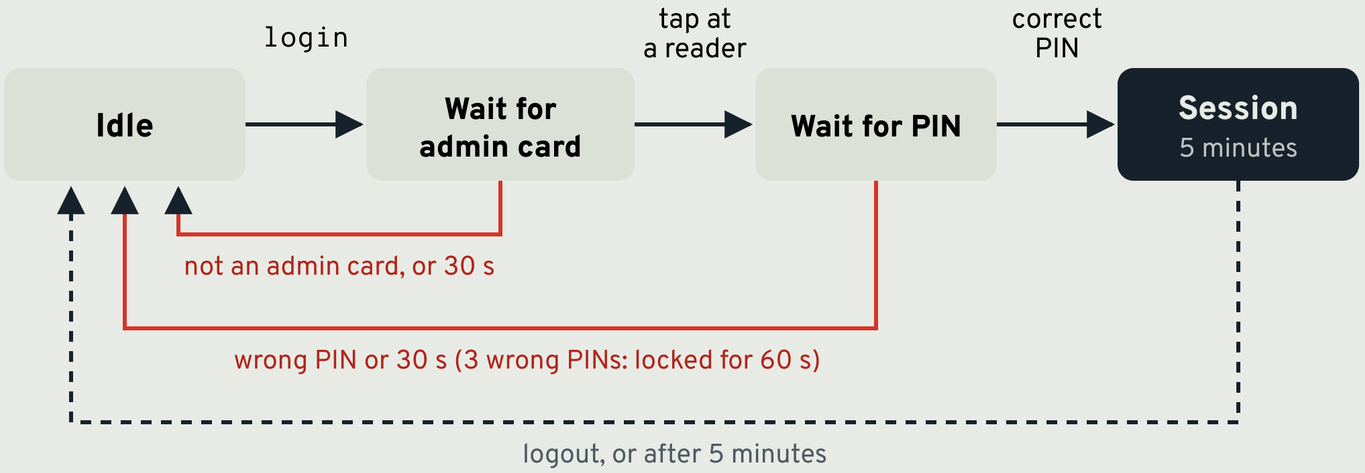
Baseline: a 360 W lamp bank left on for all opening hours. Used: the real lamp level over time.

```
cocis> rooms
Room Link   Occupancy Light  Lux  Saved Wh
1    online vacant       0%  650       0.4
2    online vacant       0%  100       0.0
```

An ENERGY line per room every 10 minutes.

Ochen's code: `room_logic.c` `room_node.c` `light_sensor.c` `actuators.c`

Changing the schedule takes a card and a PIN




```
cocis> schedule set 07:00 18:00 weekdays
Admin session required.
cocis> login
Tap the ADMIN card (Red card) at
either RFID reader within 30 s,
then type 'login <PIN>' here.
[LOG] AUTH uid=AABBCCDD result=CARD_OK
cocis> login 1234
Logged in: admin session started.
cocis> schedule set 07:00 18:00 weekdays
Schedule updated.
[LOG] CONFIG key=schedule
      value=07:00-18:00_-MTWTF-
      version=2 by=AABBCCDD
```

Something you have

The admin card, at either reader.

Something you know

A 4-digit PIN, stored only as a salted SHA-256 hash.

Everything is logged

Each step is an AUTH line; a lockout raises an alert.

The serial menu answers management's questions

Who is in the building right now?

`who`

Are the rooms lit, and what did we save?

`rooms`

Is the building open? Is anything wrong?

`status`

Has anyone edited the records?

`log verify`

Change the hours (admin only)

`schedule set 07:00 18:00 weekdays`

```
cocis> status
Time         : 2026-10-28 11:42:46 Wed (EAT)
Schedule     : 07:00-22:00_-MTWTFS (v3)
               OPEN, closes in 618 min
Lockdown     : off   Emergency: off
Inside       : 3 people
Access       : 3 granted, 0 denied, 2 alerts
Storage      : SD card OK, last seq 15
Bus          : 0 duplicate frames rejected
RFID readers: entry OK, exit OK
Admin        : logged in as AABBCCDD
Room 1       : online, vacant, light   0%, 650 lux
Room 2       : online, vacant, light   0%, 100 lux
```

Nakimuli's code: `cli/` `core_logic/auth_fsm.c` `services/auth.c` `ui/ui.c`



Tested before we demoed it

On a PC

```
$ make test
schedule  ok
access    ok
room      ok
auth      ok
types     ok

70 checks, 0 failures
```

The rules are pure C, so gcc tests them with `-Wall -Wextra -Werror`.

In Wokwi

Three scripts run the real firmware: they tap cards, move the light sliders, type commands, and fail unless the expected log lines appear.

`access.test.yaml` **pass**

`demo.test.yaml` **pass**

`rules.test.yaml` **pass**

Firmware build: 0 warnings.

Bugs we caught and fixed

A crash loop when a lamp faded on (hardware fade interrupt): now faded in software.

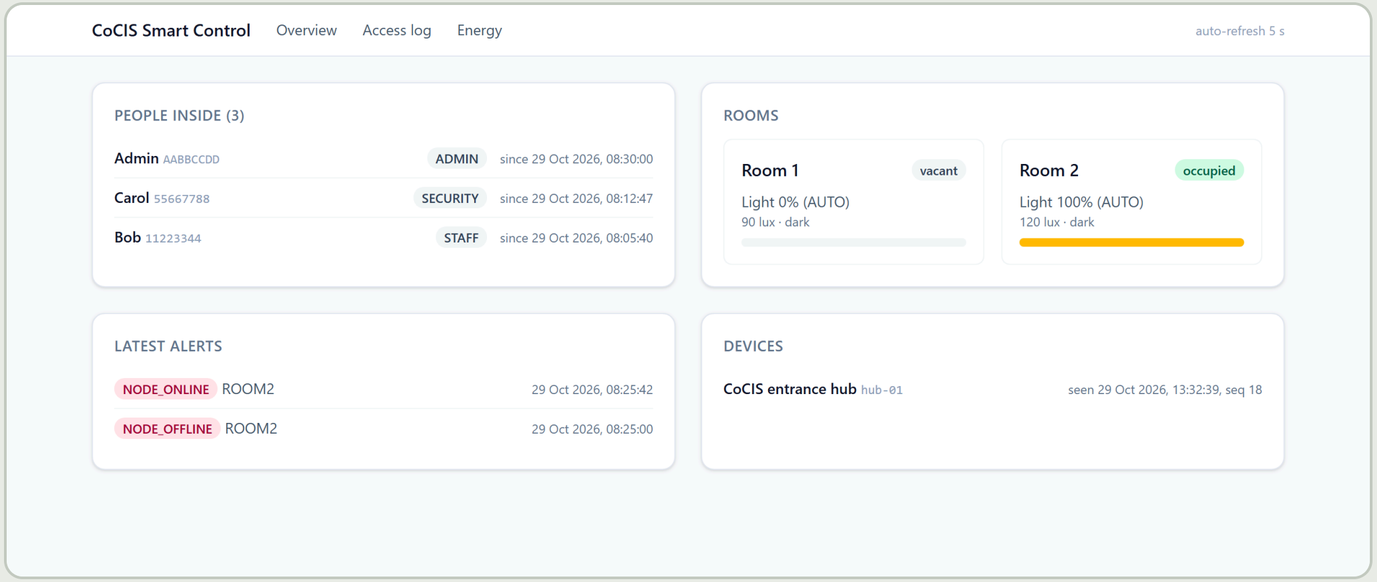
Two senders could reorder bus messages: per-sender lock.

A PIN typed after the timeout was accepted: expiry checked first.

LCD rows overflowing: every row is exactly 20 characters.



An optional dashboard that only trusts signed data



Tamara's code: `test/host/` `test/wokwi/` `dashboard/`

Signed

Every batch carries an HMAC-SHA256 signature. A bad signature or a stale timestamp gets HTTP 401.

Stored once

The same 18 events sent twice are stored once: the second run reports 18 duplicates.

Optional

Next.js and Postgres on Vercel. The building never depends on it; the hub's uploader is the next step.

Tamara Alice Nazziwa 25/U/03596/PS
Testing and cloud dashboard

What the brief asked for, and what we added

The brief

2.1	RFID entry and exit, doors, who is inside
2.2	Lights that follow occupancy
2.3	Three coordinated terminals
2.4	Records on an SD card
2.5	Serial menus for management
2.6	A schedule only authorised staff can change
2.7	Synchronisation without race conditions

Added

- Tamper-evident log
- Two-factor admin login
- Anti-passback and lockdown
- Escort window and after-hours alert
- Break-glass override
- Daylight harvesting
- Dim warning before lights off
- Energy used and saved
- Room heartbeats and offline alerts
- Keeps running without SD card or clock
- Signed cloud dashboard





Run it in your browser

wokwi.com/projects/479125502548914177

Video demonstration

youtu.be/BQb2dwnprf4

Scan to run the simulation

Press play in Wokwi, then tap a card